TECHNICAL PRIMER

FASTR for developers

A high-level map of the data, the pipeline, and the vocabulary

WHAT FASTR IS

Frequent **A**ssessments and **S**ystem **T**ools for **R**esilience — the GFF approach to rapid-cycle analytics and data use.

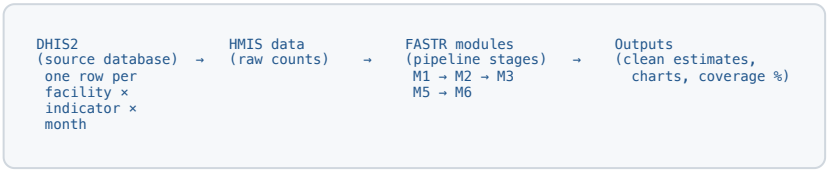
IN ONE SENTENCE

A **data pipeline** that pulls routine monthly health records out of a national database, cleans them, and turns them into trustworthy estimates of whether people are getting the care they need.

MENTAL MODEL

ETL → validate → transform → KPI. The health terms sit on top of standard data engineering — they're domain labels, nothing more.

The shape of it



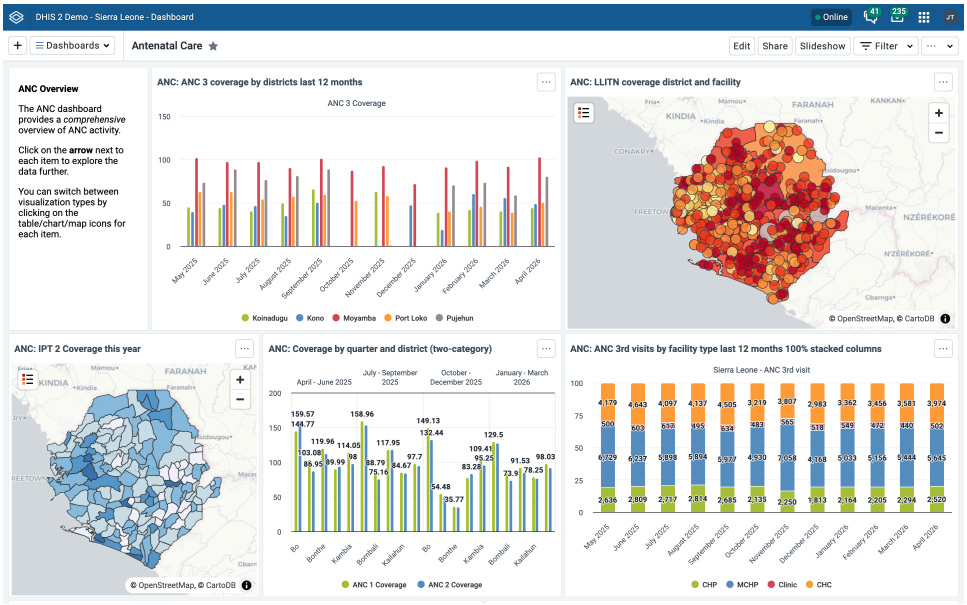
- **DHIS2** — the country's production database for health data. Web app + REST API. Read-only as far as the pipeline is concerned.
- **HMIS data** — what we pull: routine **monthly service counts**, one number per facility, per indicator, per month.
- **Modules** — pipeline stages. Each reads the previous stage's output; the dependencies form a DAG.
- **Outputs** — CSVs and charts that health teams use to make decisions.

THE DATA

Where the data comes from: DHIS2

DHIS2 is an open-source platform most low- and middle-income countries use as the national warehouse for routine health data — effectively the **source-of-truth production database**.

- Every clinic and hospital (a **facility**) reports numbers each month
- The data is keyed by **facility × indicator × month**
- It's extracted via the DHIS2 **API** (or a CSV export) into a flat table — the **HMIS data** (`hmis_XXX.csv` , where `XXX` is the country code)



A DHIS2 instance — the public demo (Sierra Leone sample data). This is the kind of system the data is pulled from.

> **Counts, not percentages.** The pipeline ingests **raw counts** — *"152 children received Penta1 at this facility in March 2024"* — never *"92% coverage."* Counts can be summed across facilities and let outlier detection work on magnitude; percentages can't and don't.

THE DATA

What an "indicator" is

In global health, an **indicator** is a defined, measurable variable used to monitor a health system — to track service delivery, coverage, or health outcomes over time and across places, and to inform decisions. It's a standardized proxy: you can't measure "is maternal health improving?" directly, so you track something countable that stands in for it.

In routine HMIS data, each indicator is a specific service event that facilities record and report monthly. (For a developer: think of it as a tracked metric — a named, counted event, like an event type in product analytics.) FASTR centres on a small core set of high-volume **RMNCAH-N** indicators (reproductive, maternal, newborn, child, adolescent health + nutrition):

ANC1 ANC4 Institutional delivery PNC1 BCG Penta1 Penta3 OPD

Indicator	Plain English
ANC1 / ANC4	A pregnant woman's 1st / 4th antenatal (pre-birth) check-up
Institutional delivery	A birth that happened in a health facility
PNC1	First postnatal (after-birth) check-up
BCG	TB vaccine, given at birth
Penta1 / Penta3	1st / 3rd dose of the 5-in-1 infant vaccine
OPD	Outpatient visits — a proxy for general use of health services

> ***Chosen for high reporting volume and because they stand in for many other services delivered at the same visit. Countries add their own on top.***

THE PIPELINE

What the modules do

Each module is a stage. They run **in order**, and each consumes the output of the ones before it — the → is a real data dependency.

1 M1 — Data quality assessment (DQA)

Validation + anomaly detection. Flags **outliers** (a count far out of line with the facility's own history) and **completeness** gaps (months a facility didn't report). Read-only — it labels problems, changes nothing.

2 M2 — Data quality adjustment

The fix stage. Consumes M1's flags and **repairs** the data — replaces outliers, fills gaps with statistical estimates — so downstream maths isn't skewed by bad inputs. Emits the adjusted dataset.

3 M3 — Service utilization

Trend analysis: are services going up or down? Uses regression to measure trends and detect **disruptions** (e.g. a drop in visits after a funding cut or a shock).

4 M5 → M6 — Coverage estimates

Coverage = what share of the people who *needed* a service actually got it. Example: 8,000 infants in a district got the measles vaccine, and about 10,000 infants live there → ~80% coverage.

The top number (8,000) is a **count** we already have. The bottom number (10,000 — the **target population**) is *not* in the data: no system reports "how many infants exist." Estimating that denominator is the hard part, and it's M5's whole job — it derives it from HMIS data, household surveys, and UN population projections. **M6** then computes the coverage % and fills the gaps between survey years.

> ***M1 validate → M2 fix → M3 trends → M5/M6 coverage. (M4 is the older coverage module being replaced by M5+M6.)***

REFERENCE

Jargon decoder

Term	Meaning
DHIS2	National health database the pipeline reads from (has an API)
HMIS	Health Management Information System — the routine data itself
Facility	A clinic or hospital — one reporting source
Indicator	A counted health event (a metric)
Count / volume	Raw number of events. The input. Never a %
Admin area	Geographic level: national → region → district → ward
Outlier	A count implausibly high vs the facility's own history
Completeness	Whether a facility reported at all in a given month
Coverage	% of the target population that received a service (a KPI)
Denominator	The target population (e.g. all pregnant women) — estimated, not counted
RMNCAH-N	The health domain: reproductive, maternal, newborn, child, adolescent health + nutrition
Disruption	A meaningful drop in service volume (shock, funding cut, etc.)
Module (Mx)	A stage in the pipeline

Going deeper

- **Methodology docs** — one page per pipeline module (DQA, adjustment, service utilization, coverage), with the full logic and parameters

  Read: FASTR-Analytics.github.io/fastr-resource-hub · Source: github.com/FASTR-Analytics/fastr-resource-hub
- **Pipeline module source** — each module (`m001` … `m006`) ships a `definition.json` (inputs, parameters, outputs) and a `script.R` (the logic)

  github.com/FASTR-Analytics/modules